

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

Кафедра Вычислительной техники

Нейронные сети

Расчетно-графическое задание

Qandle: Accelerating State Vector Simulation Using Gate-Matrix Caching and
Circuit Splitting

(Qandle: Ускорение моделирования вектора состояния с помощью
кэширования матрицы ворот и разделения цепей)

Факультет: АВТФ
Группа: АПИМ-24
Выполнил:
Разуваев В. В.

Проверил:
Гаврилов А.В.

Новосибирск, 2025 г.

Содержание

Ключевые слова:	3
Авторы:	3
Аннотация	3
Введение	4
2. Предварительные сведения	6
3. Родственные работы.....	10
4. Методы повышения производительности.....	15
5. Реализация и оценка эффективности.....	21
6. Заключение.....	25
Благодарности	26
Ссылки	27
Приложение.....	30

Ключевые слова:

Квантовые вычисления, квантовое машинное обучение, симуляция вектора состояния, гибридное машинное обучение, квантово-классическое машинное обучение, PyTorch.

Авторы:

Gerhard Stenzel, Sebastian Zielinski, Michael Kolle, Philipp Altmann, Jonas Nußlein and Thomas Gabor. Ludwig Maximilian University of Munich, Germany (Мюнхенский университет имени Людвига-Максимилиана, Германия). ICAART 2025 – 17-я Международная конференция по агентам и искусственному интеллекту. Стр. 715-723, Том 1, 23.02.2025 г.

Аннотация

Для преодоления вычислительной сложности, связанной с симуляцией вектора состояния квантовых цепей, мы предлагаем комбинацию передовых методов для ускорения их выполнения. **Кэширование матриц квантовых вентилей** снижает нагрузку от многократного применения кронекерова произведения при применении матрицы вентиля к вектору состояния за счет хранения декомпозированных частичных матриц для каждого вентиля. **Разделение цепей** делит цепь на подцепи с меньшим числом вентилей посредством построения графа зависимостей, что позволяет выполнять их параллельно или последовательно на непересекающихся подмножествах вектора состояния. Эти методы реализованы с использованием фреймворка машинного обучения PyTorch. Мы демонстрируем производительность нашего подхода, сравнивая его с другими PyTorch-совместимыми симуляторами вектора состояния. Наша реализация, названная **Qandle**, разработана для бесшовной интеграции в существующие рабочие процессы машинного обучения, предоставляя удобный API и совместимость с форматом OpenQASM. Qandle является открытым проектом, доступным на GitHub и PyPI.

Введение

Квантовое машинное обучение (QML) — это быстро развивающаяся область, направленная на объединение вычислительной мощности квантовых систем с гибкостью и масштабируемостью классических алгоритмов машинного обучения. В последние годы машинное обучение получило широкое распространение в таких областях, как распознавание изображений и речи, обработка естественного языка и рекомендательные системы. Эти приложения часто опираются на глубокое обучение, требующее значительных вычислительных ресурсов для обработки больших данных.

Квантовое машинное обучение стремится использовать потенциал квантовых вычислений для решения сложных оптимизационных задач, которые пока недоступны классическим компьютерам. Однако современные квантовые устройства сталкиваются с ограничениями: шумом оборудования, недостаточной коррекцией ошибок, ограниченной связью кубитов и их малым количеством. Эти проблемы снижают эффективность квантовых алгоритмов на практике.

Для их преодоления разрабатываются гибридные квантово-классические модели, сочетающие классические и квантовые слои. Такие модели можно обучать как на реальном оборудовании, так и на симуляторах, имитирующих поведение квантовых цепей на классических CPU/GPU. Классическая симуляция играет ключевую роль в разработке и тестировании квантовых моделей, позволяя исследователям отлаживать цепи и интегрировать их в классические ML-модели.

С ростом числа кубитов вычислительная сложность симуляции растет экспоненциально, что делает эффективность симуляторов критически важной. В данной работе представлены два метода — кэширование матриц вентилей и разделение цепей — для ускорения симуляции. Они реализованы в симуляторе Qandle, ориентированном на гибридные квантово-классические приложения в связке с PyTorch. Сравнение с существующими подходами

демонстрирует превосходство Qandle по скорости выполнения и использованию памяти.

Наши основные вклады:

1. Введение методов кэширования матриц вентилей и разделения цепей.
2. Реализация этих методов в новом симуляторе.
3. Сравнение производительности с существующими решениями.

Статья структурирована следующим образом: в Разделе 2 представлены основные понятия, в Разделе 3 — анализ существующих работ. В Разделах 4 и 5 описаны предложенные методы и их оценка. Заключение приведено в Разделе 6.

2. Предварительные сведения

2.1 Обозначения

В данной работе мы используем нотацию "старший значащий бит первый" (MSb 0) для представления квантовых состояний. В этой нотации состояние $|0000\rangle$ соответствует всем кубитам в состоянии 0, а состояние $|0001\rangle$ означает, что все кубиты находятся в состоянии 0, кроме последнего, который находится в состоянии 1. Это позволяет обеспечить последовательное и однозначное представление квантовых состояний на протяжении всего анализа.

Другие используемые обозначения включают:

- S - вектор состояния $|\phi\rangle$
- W - общее количество кубитов
- w - текущий кубит
- R_d - матричное представление вентилей для вращения вокруг оси d
- R_d - матричное представление R_d для W кубитов

2.2 Симуляция вектора состояния

Квантовое состояние $|\phi\rangle$ системы с W кубитами может быть представлено как вектор размером 2^W . Этот вектор содержит комплексные амплитуды вероятностей каждого из 2^W возможных состояний, от $|00\dots 0\rangle$ до $|11\dots 1\rangle$, полностью описывая состояние системы в любой момент времени.

Квантовые вентили, представленные унитарными матрицами, применяются к квантовому состоянию для его преобразования. На реальном квантовом оборудовании вектор состояния недоступен напрямую - его можно только приблизительно определить по вероятностным результатам измерений квантовой системы из-за присущей оборудованию шумности в эпоху NISQ.

В отличие от этого, симуляторы, работающие с полным вектором состояния, могут предоставить точное состояние системы в любой момент

времени. Однако такие симуляторы сталкиваются с проблемой при работе с большими схемами из-за экспоненциального роста вектора состояния с увеличением количества кубитов. Благодаря своей детерминированной природе симуляторы особенно хорошо подходят для создания, отладки и обучения вариационных квантовых схем.

2.3 Гибридное машинное обучение

В контексте квантового машинного обучения гибридное машинное обучение означает интеграцию классических и квантовых алгоритмов машинного обучения. Этого можно достичь путем включения обучаемых квантовых схем в более крупные модели машинного обучения или применения классических методов машинного обучения для оптимизации квантовых схем.

Обычно квантовые модели в этом контексте представляют собой вариационные квантовые схемы, состоящие из нескольких групп вентилях:

1. Слои встраивания, которые кодируют классические данные в квантовое состояние схемы. Различные методы встраивания предлагают разные компромиссы между выразительностью квантового состояния и количеством необходимых кубитов. Некоторые архитектуры схем используют методы "повторной загрузки данных" для повышения выразительности квантового состояния путем встраивания одних и тех же точек данных в нескольких местах схемы, эффективно усиливая "память" схемы о входных данных.

2. Обучаемые слои, представляющие собой параметризованные вентилях, параметры которых служат обучаемыми весами квантовой модели. Эти параметры, часто представляемые как углы вращательных вентилях, могут быть оптимизированы с помощью классических алгоритмов оптимизации, таких как градиентный спуск или его варианты.

3. Слои измерений, которые извлекают закодированную в квантовом состоянии информацию и преобразуют ее в классический выход. Этот выход может затем подвергаться дальнейшей обработке или оптимизации. Хотя

симуляторы позволяют проводить измерения в любой точке схемы, реальное квантовое оборудование обычно допускает измерения только как конечную.

Эти квантовые модели могут рассматриваться как "черные ящики", что позволяет легко интегрировать их в существующие рабочие процессы машинного обучения. Они могут применяться для широкого круга задач, включая классификацию, регрессию, кластеризацию и генеративное моделирование.

При обучении веса квантовых моделей оптимизируются с использованием таких методов, как правило сдвига параметров или классическое обратное распространение. Правило сдвига параметров позволяет вычислять градиент функции потерь без знания внутренней работы квантовой схемы, что делает его пригодным как для реального квантового оборудования, так и для симуляторов. Оно аппроксимирует градиент с помощью метода конечных разностей. С другой стороны, классическое обратное распространение, которое может быть эффективно реализовано на симуляторах вектора состояния, рассматривает квантовые и классические части модели машинного обучения отдельно и позволяет использовать разные алгоритмы оптимизации и скорости обучения, а также применять классические алгоритмы оптимизации к квантовым весам.

2.4 Концепция форм

Концепция форм используется в соответствии с понятием формы в тензорах PyTorch. Тензор представляет собой потенциально многомерную матрицу, где форма задает количество элементов или подтензоров в каждом измерении. Например, тензор с формой (2,3,4) состоит из двух подматриц, каждая из которых имеет три строки и четыре столбца, что в сумме дает $2 \cdot 3 \cdot 4$ элементов. В контексте квантовых схем квантовое состояние S системы с W кубитами может быть представлено как тензор формы 2^W , содержащий комплексные амплитуды вероятностей каждого из 2^W возможных состояний от $|00\dots 0\rangle$ до $|11\dots 1\rangle$. Это можно формализовать как комплексный вектор $S \in \mathbb{C}^{2^W}$. Используя изоморфные преобразования, мы можем изменить форму

тензора на $(d_1, d_2, \dots, d_W)$, где все d_i равны двум, а W - количество кубитов. Это изменяет представление состояния с $S \in \mathbb{C}^{2^W}$ на $S \in \mathbb{C}^{2 \times 2 \times \dots \times 2}$. Интуитивно каждое измерение этого тензора представляет кубит квантовой схемы. Например, амплитуда вероятности состояния $|010\rangle$ хранится в тензоре на позиции $(0, 1, 0)$, что соответствует первому элементу первого измерения, второму элементу второго измерения и первому элементу третьего измерения.

При применении однокубитного вентиля, представленного матрицей $G \in \mathbb{C}^{2 \times 2}$, к w -му кубиту мы можем изменить форму квантового состояния с (2^W) на $(d_0 \times d_1 \times \dots \times d_{w-1} \times d_w \times d_{w+1} \times \dots \times d_W)$ (где все измерения равны 2), а затем переупорядочить элементы в $((d_0 \times d_1 \times \dots \times d_{w-1} \times d_{w+1} \times \dots \times d_W), d_w)$. Это дает тензор формы $(2^{W-1}, 2)$, который можно умножить на матрицу вентиля G , а затем вернуть к исходной форме $S \in \mathbb{C}^{2^W}$.

В контексте машинного обучения тензор обычно расширяется дополнительным измерением, представляющим размер батча данных, увеличивая форму до $(B, 2^W)$ или $(B, 2, 2, \dots, 2)$ (где B - размер батча, например 16). Это позволяет обрабатывать несколько точек данных одновременно в течение одних и тех же прямого и обратного проходов.

3. Родственные работы

3.1 PennyLane

PennyLane – это программный фреймворк на Python 3 для дифференцируемого программирования квантовых компьютеров. Он обеспечивает поддержку широкого спектра квантового оборудования и симуляторов и легко интегрируется с библиотеками машинного обучения, такими как PyTorch и TensorFlow, а также с другими квантовыми программными платформами, включая Qiskit и Cirq. PennyLane различает квантовые и классические узлы, где квантовые узлы представляют части графа выполнения, которые работают на квантовом устройстве или симуляторе. Фреймворк предлагает обширную коллекцию квантовых операций, включая одно- и многокубитные вентили, измерения и не унитарные операции, такие как операция сброса. Кроме того, PennyLane предоставляет встроенную поддержку симуляций квантовой химии.

Производительность PennyLane в основном зависит от базовых квантовых симуляторов, причем разные реализации бэкендов предлагают различные компромиссы между вычислительной скоростью и поддерживаемыми операциями. Некоторые симуляторы даже поддерживают выполнение на GPU NVIDIA для дальнейшего повышения производительности. Для ускорения выполнения одной и той же квантовой схемы с разными параметрами PennyLane использует методы кэширования. Однако эти кэши эффективны только при многократном выполнении одной и той же схемы с одинаковыми параметрами. Кроме того, большинство вентиля и симуляторов поддерживают батчинг (хотя и не все), что является распространенной техникой в машинном обучении. Разделение схем в PennyLane позволяет выполнять части схемы независимо, что дает возможность запускать большие схемы на меньшем оборудовании. Однако этот процесс сопровождается значительными накладными расходами по времени симуляции и использованию памяти. PennyLane также предлагает

методы визуализации схем и поддерживает импорт и экспорт схем в формате OpenQASM 2.0.

3.2 Qiskit

Qiskit – это комплексный фреймворк для квантовых вычислений, разработанный IBM. Он предлагает широкий спектр квантовых операций, включая одно- и многокубитные вентили, измерения и не унитарные операции. Qiskit предоставляет доступ к реальному квантовому оборудованию через IBMQ Experience, позволяя пользователям запускать свои квантовые схемы на квантовых компьютерах IBM или симуляторах в облаке. Локальные симуляторы также доступны без необходимости регистрации.

Для оптимизации квантовых схем под конкретные квантовые устройства Qiskit предлагает транспайлер. Транспайлер адаптирует схему к аппаратно-специфичным ограничениям связи, которые определяют допустимые комбинации кубитов для вентилей CNOT и их направления. Он также обрабатывает ограничения на вентили, разлагая неподдерживаемые вентили на поддерживаемый набор вентилей для целевого оборудования. Кроме того, используются методы слияния вентилей и их отмены для уменьшения общего количества вентилей, что приводит к улучшению времени выполнения и снижению влияния аппаратного шума и ошибок.

Важно отметить, что Qiskit использует младший значащий бит как первый бит (LSb 0), в то время как большинство других фреймворков используют старший значащий бит как первый бит (MSb 0). Это различие может привести к путанице при одновременном использовании нескольких фреймворков.

Интеграция Qiskit с IBMQ Experience предоставляет исследователям и разработчикам ценные ресурсы для изучения и экспериментов с квантовыми вычислениями. Сочетание обширных квантовых операций, возможностей транспайлера и доступа к реальному квантовому оборудованию делает Qiskit мощным инструментом для разработки и выполнения квантовых алгоритмов.

3.3 TorchQuantum

TorchQuantum - это недавно разработанный фреймворк на основе PyTorch, ориентированный на скорость выполнения и параллелизацию. Он обеспечивает легкую интеграцию с Qiskit от IBM, позволяя легко преобразовывать свои модели в схемы Qiskit. Эти схемы затем могут быть выполнены на реальном квантовом оборудовании с использованием IBMQ или экспортированы в формат OpenQASM.

TorchQuantum использует распределенные вычисления на GPU для обработки крупномасштабных схем и размеров батчей, что приводит к значительному улучшению производительности по сравнению с PennyLane. Фактически, TorchQuantum демонстрирует улучшение времени выполнения до 1000 раз. Фреймворк наследует поддержку обратного распространения и батчинга из библиотеки PyTorch, обеспечивая эффективное масштабирование с увеличением количества кубитов и размера батча.

Одной из примечательных особенностей TorchQuantum является его дизайн как инструмента для запуска QuantumNAS - шум-адаптивного поиска устойчивых квантовых схем. Это достигается путем разделения схем на меньшие подсхемы и их независимой оптимизации. Подсхемы затем комбинируются с использованием эволюционного алгоритма. Такой подход минимизирует влияние аппаратного шума и, следовательно, максимизирует производительность на реальном квантовом оборудовании, что делает его чрезвычайно полезным для приложений квантового машинного обучения.

3.4 Вклад

Наш вклад заключается в предложении и комбинации передовых методов, направленных на ускорение выполнения квантовых схем. В результате мы разработали высокопроизводительный симулятор вектора состояния под названием Qandle, который обеспечивает плавную интеграцию в рабочие процессы машинного обучения на основе PyTorch. Qandle демонстрирует значительные улучшения во времени выполнения и

использовании памяти по сравнению с существующими фреймворками, такими как PennyLane, Qiskit и TorchQuantum. Примечательно, что оба наших метода основаны на матрицах, что делает их высоко совместимыми с функцией `torch.compile` PyTorch, обеспечивая дополнительное повышение производительности.

Важно подчеркнуть, что наш симулятор не предназначен для замены PennyLane или Qiskit. Вместо этого он служит ценным инструментом для приложений квантового машинного обучения в экосистеме PyTorch, аналогично TorchQuantum. Наш симулятор уделяет приоритетное внимание эффективному выполнению квантовых схем на платформах CPU и GPU, делая акцент на производительности, а не на предоставлении продвинутых инструментов визуализации или прямого доступа к квантовому оборудованию, в отличие от более зрелых фреймворков, таких как PennyLane, Qiskit и TorchQuantum.

Используя представленные методы кэширования матриц вентилей (Раздел 4.1) и декомпозиции частичных матриц (Раздел 4.2), наш симулятор оптимизирует выполнение операций вентилей над вектором состояния. Это приводит к уменьшению вычислений (Раздел 5.2) и требований к памяти (Раздел 5.3) во время прямого прохода квантовой схемы.

Интеграция нашего симулятора с PyTorch позволяет легко включать квантовые схемы в модели машинного обучения. Это позволяет исследователям и практикам изучать потенциал квантовых вычислений в различных областях, таких как симуляции квантовой химии, задачи оптимизации и генеративное моделирование. Кроме того, совместимость нашего симулятора с форматом OpenQASM обеспечивает взаимодействие с другими квантовыми программными платформами, позволяя легко интегрироваться с существующими квантовыми алгоритмами и библиотеками благодаря удобному, но мощному API (Раздел 5.1).

В заключение, мы объединяем представленные методы кэширования матриц вентилей и разделения схем в нашем высокопроизводительном

симуляторе вектора состояния Qandle, с уменьшенным использованием памяти и увеличенной скоростью выполнения, а также поддержкой компиляции "на лету", что делает его привлекательным выбором для исследователей и практиков, стремящихся использовать возможности квантовых вычислений в своих рабочих процессах машинного обучения.

4. Методы повышения производительности

4.1 Кэширование матриц квантовых вентилей

Для улучшения времени выполнения мы применяем метод, называемый **кэшированием матриц квантовых вентилей**, который заключается в хранении частичных матриц вентилей. Эти частичные матрицы представляют собой декомпозиции матриц вентилей на две матрицы одинаковой формы, но с более высокой степенью разреженности. Например, мы можем разложить вентиль $R_x(\theta)$ на две матрицы R_{xa} и R_{xb} , обе формы $(2,2)$, но содержащие только по два ненулевых элемента каждая.

Декомпозиция матрицы вентилей выполняется следующим образом:

$$\begin{aligned} R_x(\theta) &= \begin{bmatrix} \cos(\theta/2) & -i \sin(\theta/2) \\ -i \sin(\theta/2) & \cos(\theta/2) \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \cdot \cos(\theta/2) + \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \cdot -i \sin(\theta/2) \\ &= R_{xa} \cdot \cos(\theta/2) + R_{xb} \cdot -i \sin(\theta/2) \end{aligned} \tag{1}$$

Преимущество использования этих частичных матриц заключается в том, что они требуют меньше операций во время прямого прохода. Вместо выделения памяти и заполнения полной матрицы вентилей мы можем просто умножить параметры на соответствующие частичные матрицы, сложить результаты, а затем умножить на вектор состояния. Это уменьшает вычислительную сложность и повышает общую эффективность схемы.

Кроме того, преимущества кэширования матриц вентилей становятся еще более заметными при работе со схемами, содержащими несколько кубитов. Для расширения матрицы вентилей R_x до полного размера вектора состояния мы вычисляем кронекерово произведение ($\otimes$) частичных матриц R_{xa} и R_{xb} с единичными матрицами, получая $R_x \in \mathbb{C}^{(2^W \times 2^W)}$:

$$\begin{aligned}
\mathcal{R}_x(\theta) &= I_{2^w} \otimes R_x(\theta) \otimes I_{2^{(W-w)}} \\
&= I_{2^w} \otimes R_{xa} \otimes I_{2^{(W-w)}} \cdot \cos(\theta/2) \\
&\quad + I_{2^w} \otimes R_{xb} \otimes I_{2^{(W-w)}} \cdot -i \sin(\theta/2) \\
&= \mathcal{R}_{xa} \cdot \cos(\theta/2) + \mathcal{R}_{xb} - i \sin(\theta/2)
\end{aligned} \tag{2}$$

Крайне важно соблюдать правильный порядок выполнения кронекерова произведения относительно количества состояний 2^w для кубитов перед вентилем и $2^{(W-w)}$ для кубитов после вентилей. Этот порядок важен для правильного преобразования формы вектора состояния после применения вентилей. Используя кэшированные частичные матрицы R_{xa} и R_{xb} , применение расширенных матриц R_x выполняется быстрее, чем вычисление полной матрицы вентилей для каждого прямого прохода, что потребовало бы многократного применения кронекерова произведения.

Кэширование матриц вентилей не ограничивается однокубитными вентилями, но также распространяется на многокубитные вентили, такие как CNOT, и составные структуры вентилей, такие как вращательные вентили для слоев углового встраивания. Для них каждый вращательный вентиль декомпозируется на две частичные матрицы. Для удобства доступа и лучшего кэширования на аппаратном уровне две группы частичных матриц R_a и R_b объединяются в тензоры формы $(W, 2^w, 2^w)$. При встраивании частичные функции встраивания (например, $f_{ax}(\theta) = \cos(\theta/2)$ и $f_{bx}(\theta) = -i \cdot \sin(\theta/2)$ для вентилей R_x) вычисляются для всех входных данных, давая в результате два вектора формы (W) . Эти векторы затем умножаются на частичные матрицы R_a и R_b соответственно вдоль первой оси. Полученные матрицы складываются вместе, формируя полную последовательность матриц вентилей для слоя встраивания, которые теперь можно умножать на вектор состояния.

Вычислительно затратные части операции встраивания, такие как многократное применение кронекерова произведения, выполняются только один раз во время инициализации схемы и кэшируются для последующего использования. Хотя кэш работает вычислительно быстро, он становится

ресурсоемким по памяти с увеличением количества кубитов. Для смягчения этой проблемы мы применяем разделение схем (см. раздел 4.2).

Хотя эти матрицы в основном состоят из нулей (матрица для W кубитов содержит $2^{(2W)} - 2^W$ нулей), было бы выгодно использовать разреженные матричные представления, которые быстрее умножаются. Однако наши предварительные тесты показали, что из-за постоянных умножений с квантовым состоянием (которое представляет собой очень плотный вектор) и необходимых преобразований типов, накладные расходы значительно перевешивают преимущества разреженности. Поэтому Qandle не использует разреженные матрицы для матриц вентиляей.

PennyLane, с другой стороны, использует агрессивный подход к кэшированию, при котором структура схемы, входные данные и результаты сохраняются в кэше, причем структура и входные данные выступают в качестве ключей. Эта стратегия кэширования позволяет добиться быстрого времени выполнения при повторных выполнениях одной и той же схемы, особенно когда количество вентиляей и кубитов невелико. Однако с увеличением количества кубитов кэш становится менее эффективным. Во многих приложениях квантового машинного обучения входные данные изменяются с каждым прямым проходом, что приводит к частым промахам кэша. Это еще больше снижает преимущества механизма кэширования PennyLane. Влияние кэширования PennyLane можно наблюдать в сравнении скорости выполнения, представленном на Рисунке 1.

4.2 Разделение квантовых систем

Одной из основных проблем, с которыми сталкиваются симуляторы вектора состояния, является экспоненциальный рост вектора состояния и соответствующего размера матриц вентиляей с увеличением количества кубитов. При увеличении количества кубитов, обозначаемого как W , размер вектора состояния схемы становится 2^W , а матрицы вентиляей, участвующие в вычислениях, достигают размера $2^W \times 2^W$. Следовательно, реализации

квантовых схем, основанные на наивном умножении вектора состояния и матриц вентилей, с трудом справляются с большими схемами.

Для решения этой проблемы вычислительной сложности мы предлагаем метод, называемый разделением схем. Идея разделения схем заключается в разбиении схемы на меньшие подсхемы, тем самым уменьшая размеры матриц и требуемые память и время вычислений. Это разделение может быть выполнено во время создания схемы, устраняя необходимость выбирать между качеством разделения и временем выполнения. Разделенные схемы, которые по сути являются группами квантовых вентилей, могут затем выполняться последовательно, работая только с подмножеством полного вектора состояния в каждый момент времени.

Для формирования этих групп мы интерпретируем схему как граф зависимостей, где каждый вентиль CNOT представляет узел, игнорируя другие вентиля. В этом графе два вентиля CNOT соединены ребром, если они используют один и тот же управляющий или целевой кубит и следуют друг за другом в схеме. В настоящее время наша реализация использует простой жадный алгоритм. Он перебирает все поддеревья графа зависимостей и создает новую группу, когда текущая группа превысит заданное максимальное количество кубитов (обычно от трех до шести). На последнем этапе ранее игнорируемые однокубитные вентиля добавляются в ближайшую группу вентилей CNOT на том же кубите.

Исходная большая схема была разложена на меньшие подсхемы, которые можно рассматривать как унитарные вентиля, действующие на несколько кубитов. Во время выполнения схемы вектор состояния преобразуется по форме в соответствии с размерами подсхемы. После применения подсхемы вектор состояния возвращается к исходным размерам. В процессе преобразования формы кубиты, задействованные в подсхеме, хранятся в отдельном измерении. Например, если схема имеет пять кубитов с метками 0,1,2,3,4, а подсхема действует на кубиты 1 и 2, преобразование

формы изменит вектор состояния с размерности (2^5) на размерность ($d_0 \times d_3 \times d_4, d_1 \times d_2$). Это позволяет выполнять матричное умножение между подсхемой (с матрицей вентиля $G \in \mathbb{C}^{(2^2 \times 2^2)}$) и состояниями по последнему измерению. В случае выполнения батча дополнительное измерение батча вектора состояния объединяется во время преобразования формы, при этом сохраняется исходный размер батча b для обратного преобразования. В результате преобразованный вектор состояния имеет размерность ($b \times d_0 \times d_3 \times d_4, d_1 \times d_2$) для выполнения батча. Накладные расходы, связанные с этим процессом преобразования формы, оказывают незначительное влияние на скорость выполнения по сравнению с вычислительной нагрузкой матричных умножений. Кроме того, аппаратное кэширование не затрагивается, так как батчи обрабатываются независимо.

4.3 Дополнительные оптимизации

Для повышения качества процесса машинного обучения мы применяем методы переотображения квантовых весов. В процессе переотображения все квантовые веса преобразуются в новый диапазон, например $[-\pi, \pi]$, с использованием гладких функций, таких как гиперболический тангенс ($\tanh$). Дополнительные вычислительные затраты на этап переотображения незначительны по сравнению с многочисленными другими операциями, выполняемыми во время каждого прямого прохода. Однако это дает заметные улучшения в процессе обучения, включая более быструю сходимость и более стабильную кривую потерь.

Кроме того, мы рекомендуем использовать функцию `torch.compile` PyTorch для дальнейшей оптимизации выполнения нашего симулятора. Поскольку наша реализация полностью основана на тензорных операциях PyTorch, она может быть скомпилирована в единый граф выполнения. Этот процесс компиляции позволяет ускорить выполнение как на CPU, так и на GPU за счет оптимизации графа выполнения. Эта оптимизация включает переупорядочивание порядка выполнения параллелизуемых операций для

улучшения размещения в аппаратном кэше и объединение последовательных операций преобразования формы в одну операцию. Уменьшая количество обращений к системной памяти и CPU-циклов, процесс компиляции может значительно повысить общую производительность нашего симулятора.

5. Реализация и оценка эффективности

5.1 API

Мы демонстрируем предложенные методы через реализацию PyTorch-совместимого симулятора вектора состояния. Наш симулятор разработан для обеспечения совместимости с другими квантовыми программными платформами, включая простой экспорт в формат OpenQASM. Кроме того, мы предоставляем интуитивно понятный API, который тесно интегрируется со стандартным API PyTorch. Такой подход позволяет легко встраивать наши квантовые схемы как `torch.nn.Modules` в существующие рабочие процессы машинного обучения. Подобно традиционным модулям PyTorch (например, сверточным слоям), мы храним квантовые веса как параметры, устраняя необходимость ручного управления квантовыми весами и их градиентами, что требуется в PennyLane. При этом пользователи сохраняют возможность ручного доступа и модификации весов через метод `parameters` модуля.

5.2 Время выполнения

Для оценки времени выполнения предложенных методов в нашем симуляторе мы сравниваем его с временем выполнения PennyLane, Qiskit и TorchQuantum. Для PennyLane и Qiskit, которые предлагают несколько бэкендов, мы выбираем самый быстрый доступный бэкенд для каждого (определяется предварительным тестированием).

Чтобы обеспечить точные измерения, мы выполняем разогревочные прогоны, позволяющие компилировать модули по требованию (just-in-time), после чего они сохраняются в системной памяти. Используются случайные входные данные для имитации выполнения большого набора данных, превышающего объем кэшей CPU и системной памяти. Веса квантовой схемы модифицируются с помощью классического оптимизатора. Для минимизации влияния других компонентов используются тривиальная функция потерь и хорошо зарекомендовавший себя оптимизатор Adam.

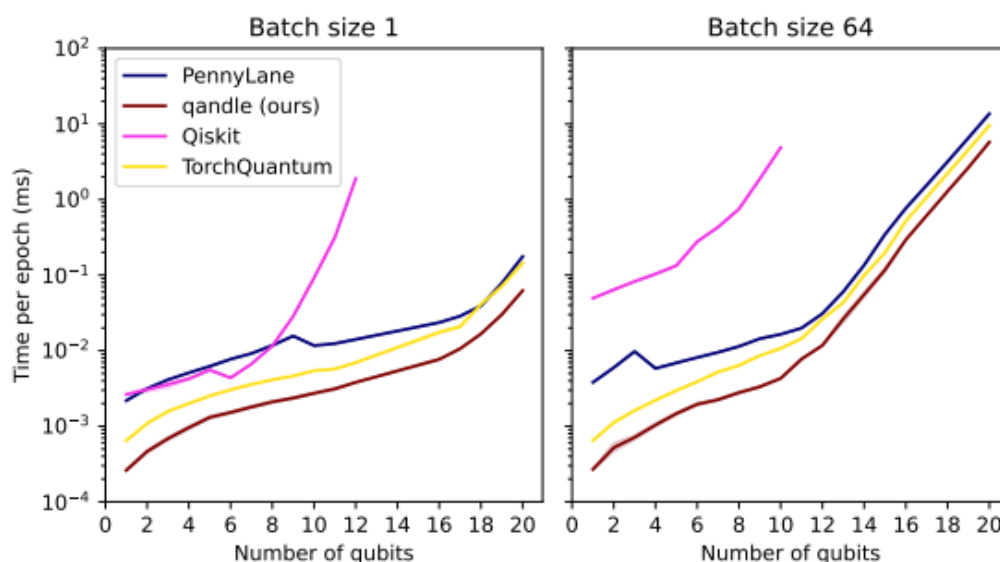


Рисунок 1: Результаты симуляции для сети

Результаты оценки времени выполнения (среднее из 15 запусков, другие статистики показаны на Рисунке 3) на Рисунке 1 демонстрируют превосходство Qandle по сравнению с другими симуляторами. Qandle последовательно превосходит TorchQuantum, который специально разработан для высокой скорости выполнения.

Кривая скорости показывает влияние механизма кэширования PennyLane. С ростом количества кубитов время выполнения увеличивается до определенного момента, определяемого размером батча, после чего функция кэширования отключается. В этот момент время выполнения временно снижается, прежде чем снова начать расти. Такое поведение обусловлено нашей экспериментальной установкой, где для каждого прямого прохода используются разные входные данные (случайная выборка) и веса (модифицированные оптимизатором), что приводит к промахам кэша. В сценариях, где одна и та же схема выполняется многократно без изменений входных данных или весов (например, для наборов данных, помещающихся в размер батча, или во время вывода), механизм кэширования PennyLane обеспечил бы лучшую производительность, чем наблюдалось в этой оценке. Однако мы считаем, что такой сценарий не является реалистичным для обучения моделей квантового машинного обучения.

5.3 Использование памяти

Для оценки использования памяти мы выполнили те же схемы на разных симуляторах и измерили их пиковое использование памяти. Мы использовали реалистичный сценарий обучения, выполняя несколько обратных проходов с простой функцией потерь и изменяющимися входными данными, чтобы избежать эффектов кэширования. Мы измерили максимальный резидентный размер набора (RSS) процесса Python, включая загруженные библиотеки симуляторов и библиотеку PyTorch, с помощью команды GNU time. Каждое измерение повторялось 15 раз с незначительной вариативностью, вызванной подкачкой страниц и другими системными процессами. Все тесты проводились на рабочих станциях с 64 ГБ оперативной памяти и процессорами Intel Core i9-9900. Симуляторы с несколькими бэкендами (например, PennyLane и Qiskit) запускались с их самыми быстрыми вариантами бэкендов: `default.qubit.torch` и `statevector_simulator` на симуляторе Aer соответственно.

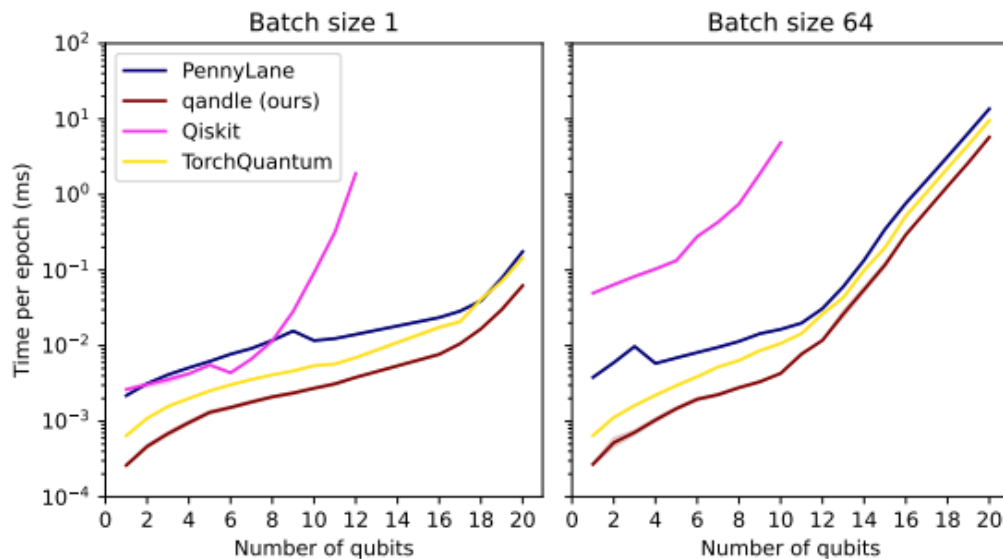


Рисунок 2: Использование памяти для аппаратно-эффективной схемы SU(2) с различным количеством кубитов

Поведение масштабирования памяти демонстрирует характеристики, аналогичные другим симуляторам: даже с оптимизациями использование памяти растет экспоненциально с увеличением количества кубитов. Это связано с большим размером вектора состояния, состоящего из 2^W комплексных чисел, и соответствующими накладными расходами памяти для

матричных умножений. Для протестированных квантовых схем с числом кубитов до 20 (см. Рисунок 2 для реализации аппаратно-эффективной схемы $SU(2)$ на всех кубитах) наш симулятор демонстрирует меньшее использование памяти по сравнению с другими симуляторами, хотя все равно масштабируется экспоненциально с ростом числа кубитов. TorchQuantum и PennyLane показывают схожую производительность (с небольшим преимуществом TorchQuantum), в то время как Qiskit использует больше всего памяти, что может сделать его непригодным для очень больших схем.

6. Заключение

В данной статье представлены передовые методы кэширования матриц квантовых вентилях и разделения квантовых схем для ускорения выполнения квантовых цепей. Наша реализация этих методов, симулятор Qandle, представляет собой высокопроизводительный симулятор вектора состояния, который обеспечивает бесшовную интеграцию с рабочими процессами машинного обучения на основе PyTorch. Qandle демонстрирует значительные улучшения во времени выполнения и использовании памяти по сравнению с существующими фреймворками, такими как PennyLane, Qiskit и TorchQuantum, что подтверждает эффективность предложенных методов. Кроме того, совместимость Qandle с функцией `torch.compile` PyTorch обеспечивает дополнительное повышение производительности. Удобный API Qandle делает его доступным даже для пользователей с ограниченным опытом в квантовом машинном обучении и квантовых вычислениях, расширяя тем самым доступность этой технологии для более широкой аудитории.

Основываясь на впечатляющих результатах предложенных методов, мы рекомендуем их внедрение в другие существующие симуляторы.

В качестве будущей работы мы планируем:

Расширить набор поддерживаемых квантовых вентилях, особенно многокубитных, таких как вентиль Тоффли, что позволит симулировать более сложные схемы.

Разработать более совершенный алгоритм разделения схем на основе графовых алгоритмов, использующий граф зависимостей схемы. Этот алгоритм будет определять оптимальное разделение, уменьшая количество подсхем и минимизируя накладные расходы на преобразование вектора состояния при сохранении эффективного выполнения. Для этой цели мы предлагаем исследовать методы раскраски графов или алгоритмы декомпозиции разделения.

Благодарности

Данная работа была частично поддержана Федеральным министерством образования и научных исследований Германии в рамках программы финансирования "квантовые технологии — от фундаментальных исследований до рынка" (номер контракта: 13N16196).

Ссылки

1. Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mane, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viegas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., and Zheng, X. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. Software available from tensorflow.org.
2. Bauckhage, C., Bye, R., Knopf, C., Mustafic, M., Piatkowski, N., Reese, B., Stahl, R., and Sultanow, E. (2022). Quantum machine learning in the context of its security.
3. Bergholm, V., Izaac, J., Schuld, M., Gogolin, C., Ahmed, S., Ajith, V., Alam, M. S., Alonso-Linaje, G., AkashNarayanan, B., Asadi, A., Arrazola, J. M., Azad, U., Banning, S., Blank, C., Bromley, T. R., Cordier, B. A., Ceroni, J., Delgado, A., Matteo, O. D., Dusko, A., Garg, T., Guala, D., Hayes, A., Hill, R., Ijaz, A., Isacsson, T., Ittah, D., Jahangiri, S., Jain, P., Jiang, E., Khandelwal, A., Kottmann, K., Lang, R. A., Lee, C., Loke, T., Lowe, A., McKiernan, K., Meyer, J. J., Montanez-Barrera, J. A., Moyard, R., Niu, Z., O’Riordan, L. J., Oud, S., Panigrahi, A., Park, C.-Y., Polatajko, D., Quesada, N., Roberts, C., Sa, N., Schoch, I., Shi, B., Shu, S., Sim, S., Singh, A., Strandberg, I., Soni, J., Szava, A., Thabet, S., Vargas-Hernandez, R. A., Vincent, T., Vitucci, N., Weber, M., Wierichs, D., Wiersema, R., Willmann, M., Wong, V., Zhang, S., and Killoran, N. (2022). PennyLane: Automatic differentiation of hybrid quantum-classical computations.
4. Cerezo, M., Arrasmith, A., Babbush, R., Benjamin, S. C., Endo, S., Fujii, K., McClean, J. R., Mitarai, K., Yuan, X., Cincio, L., and Coles, P. J. (2021). Variational quantum algorithms. *Nature Reviews Physics*, 3(9):625–644.

5. Cross, A. W., Bishop, L. S., Smolin, J. A., and Gambetta, J. M. (2017). Open quantum assembly language.
6. Farhi, E., Goldstone, J., Gutmann, S., and Zhou, L. (2022). The Quantum Approximate Optimization Algorithm and the Sherrington-Kirkpatrick Model at Infinite Size. *Quantum*, 6:759.
7. Kingma, D. P. and Ba, J. (2017). Adam: A method for stochastic optimization.
8. Kolle, M., Giovagnoli, A., Stein, J., Mansky, M. B., Hager, J., and Linnhoff-Popien, C. (2023a). Improving convergence for quantum variational classifiers using weight re-mapping.
9. Kolle, M., Giovagnoli, A., Stein, J., Mansky, M. B., Hager, J., Rohe, T., Muller, R., and Linnhoff-Popien, C. (2023b). Weight re-mapping for variational quantum algorithms.
10. Kolle, M., Stenzel, G., Stein, J., Zielinski, S., Ommer, B., and Linnhoff-Popien, C. (2024). Quantum denoising diffusion models.
11. Nielsen, M. A. and Chuang, I. L. (2001). Quantum computation and quantum information. *Phys. Today*, 54(2):60.
12. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems* 32, pages 8024–8035. Curran Associates, Inc.
13. Preskill, J. (2018). Quantum computing in the NISQ era and beyond. *Quantum*, 2:79.
14. Qiskit contributors (2023). Qiskit: An open-source framework for quantum computing.
15. Rebentrost, P., Mohseni, M., and Lloyd, S. (2014). Quantum support vector machine for big data classification. *Physical review letters*, 113(13):130503.
16. Schuld, M. and Petruccione, F. (2021). *Machine learning with quantum computers*. Springer.

17. Stamatopoulos, N., Egger, D. J., Sun, Y., Zoufal, C., Iten, R., Shen, N., and Woerner, S. (2020). Option pricing using quantum computers. *Quantum*, 4:291.
18. Wang, H., Ding, Y., Gu, J., Li, Z., Lin, Y., Pan, D. Z., Chong, F. T., and Han, S. (2022). Quantumnas: Noise-adaptive search for robust quantum circuits. In *The 28th IEEE International Symposium on HighPerformance Computer Architecture (HPCA-28)*.
19. Wille, R., Van Meter, R., and Naveh, Y. (2019). IBM’s qiskit tool chain: Working with and developing for real quantum computers. In *2019 Design, Automation and Test in Europe Conference and Exhibition (DATE)*, pages 1234–1240.
20. Zoufal, C., Lucchi, A., and Woerner, S. (2019). Quantum generative adversarial networks for learning and loading random distributions. *npj Quantum Information*, 5(1):103.

Приложение

Время выполнения

На Рисунке 3 представлены минимальные времена выполнения симуляторов для тех же схем при полном прямом и обратном проходе. Результаты согласуются со средним временем выполнения, показанным на Рисунке 1, где Qandle демонстрирует наилучшую производительность, за ним следуют TorchQuantum и PennyLane. Минимальное время выполнения сильнее подвержено влиянию других системных процессов и механизмов кэширования, поэтому эти результаты менее воспроизводимы.

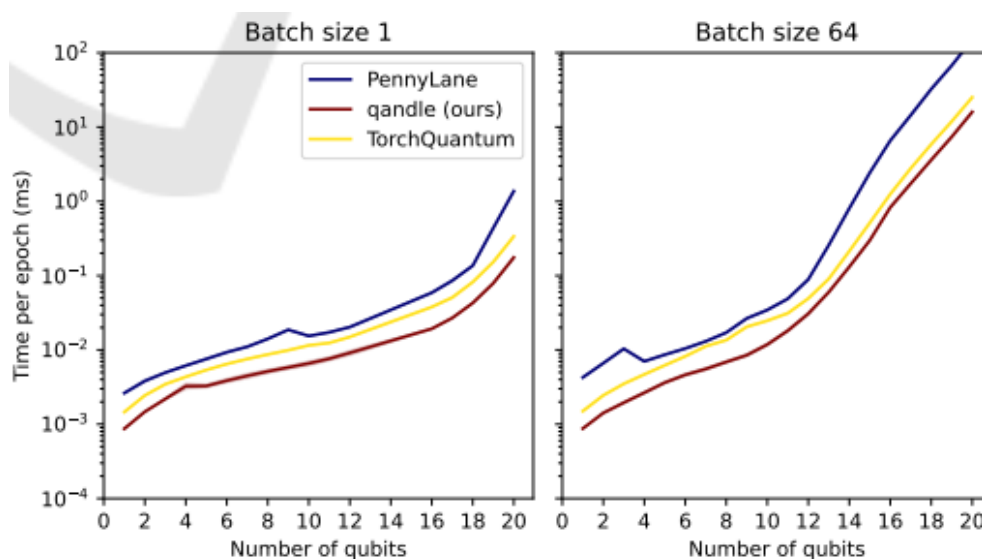


Рисунок 3: Результаты симуляции для сети, показывающие только самый быстрый прогон